

CStats - Lab #10: Auto-Encoders

May 12, 2026

Reminder

An auto-encoder uses an encoder and a decoder:

$$x \xrightarrow{\text{encoder } f_\theta} z \xrightarrow{\text{decoder } g_\phi} \hat{x}.$$

The training target is the input itself. A common objective is

$$\min_{\theta, \phi} \frac{1}{n} \sum_{i=1}^n \|x_i - g_\phi(f_\theta(x_i))\|_2^2.$$

The latent vector z is also called an embedding, representation, code, or bottleneck.

Exercise 1.

Consider an auto-encoder for input vectors $x \in \mathbb{R}^6$. The encoder maps the input to a 2-dimensional latent code, and the decoder maps the code back to a 6-dimensional reconstruction:

$$z = f_\theta(x) = \sigma(W_e x + b_e), \quad \hat{x} = g_\phi(z) = \sigma(W_d z + b_d).$$

Here σ is applied element by element.

- (a) What are the shapes of W_e , b_e , W_d , and b_d ?
- (b) Write the reconstruction loss for one data point.
- (c) Write the average reconstruction loss for a dataset $x_1, \dots, x_n$.
- (d) Why is this called self-supervised learning?
- (e) Why can the latent z be useful for downstream tasks?
- (f) Why is the latent z not robust for downstream tasks?

Solution 1.

(a) Since $x \in \mathbb{R}^6$ and $z \in \mathbb{R}^2$, the encoder weight matrix must map 6 numbers to 2 numbers:

$$W_e \in \mathbb{R}^{2 \times 6}, \quad b_e \in \mathbb{R}^2.$$

Since $z \in \mathbb{R}^2$ and $\hat{x} \in \mathbb{R}^6$, the decoder weight matrix must map 2 numbers to 6 numbers:

$$W_d \in \mathbb{R}^{6 \times 2}, \quad b_d \in \mathbb{R}^6.$$

(b) For one data point, a common reconstruction loss is

$$L(x, \hat{x}) = \|x - \hat{x}\|_2^2 = \sum_{j=1}^6 (x_j - \hat{x}_j)^2.$$

Sometimes we divide by 6 and call it the mean squared error. This changes the scale, but not the minimizer.

(c) For a dataset,

$$J(\theta, \phi) = \frac{1}{n} \sum_{i=1}^n \|x_i - g_\phi(f_\theta(x_i))\|_2^2.$$

(d) It is self-supervised because we do not need an external label such as a class name. The model creates its own target: the target for input x is again x .

(e) The code z is forced to keep information that helps reconstruct the input. If the bottleneck is small, the model cannot simply copy every coordinate. It must learn a compressed representation of useful patterns. This representation can then be used as features for classification, clustering, visualization, anomaly detection, or other downstream tasks.

(f) z does not accurately model a whole distribution; it is data-specific so generalizes badly.

Exercise 2.

This exercise shows a very small bottleneck by hand. Suppose the input is two-dimensional, $x = (x_1, x_2)$, and the auto-encoder uses a one-dimensional code:

$$z = \frac{x_1 + x_2}{2}, \quad \hat{x} = (z, z).$$

Use the squared reconstruction loss

$$L(x) = \|x - \hat{x}\|_2^2.$$

Consider the three data points

$$x^{(A)} = (3, 3), \quad x^{(B)} = (4, 0), \quad x^{(C)} = (0, 4).$$

- (a) Compute z and $\hat{x}$ for each data point.
- (b) Compute the reconstruction loss for each data point.
- (c) Which point is reconstructed perfectly?
- (d) What kind of data pattern does this simple auto-encoder preserve?

Solution 2.

(a) For $x^{(A)} = (3, 3)$,

$$z = \frac{3+3}{2} = 3, \quad \hat{x}^{(A)} = (3, 3).$$

For $x^{(B)} = (4, 0)$,

$$z = \frac{4+0}{2} = 2, \quad \hat{x}^{(B)} = (2, 2).$$

For $x^{(C)} = (0, 4)$,

$$z = \frac{0+4}{2} = 2, \quad \hat{x}^{(C)} = (2, 2).$$

(b) The losses are

$$\begin{aligned} L(x^{(A)}) &= (3-3)^2 + (3-3)^2 = 0, \\ L(x^{(B)}) &= (4-2)^2 + (0-2)^2 = 4 + 4 = 8, \\ L(x^{(C)}) &= (0-2)^2 + (4-2)^2 = 4 + 4 = 8. \end{aligned}$$

(c) The point $(3, 3)$ is reconstructed perfectly.

(d) This auto-encoder preserves the average intensity and reconstructs points on the diagonal line $x_1 = x_2$. It loses information about the difference $x_1 - x_2$. Therefore, it treats diagonal points as normal and off-diagonal points as harder to reconstruct. This is the main idea of a bottleneck: keep the information that fits the learned low-dimensional pattern, and discard the rest.

Exercise 3.

Now consider the smallest possible linear auto-encoder:

$$z = w_e x, \quad \hat{x} = w_d z, \quad L = \frac{1}{2}(\hat{x} - x)^2.$$

Here x , z , and $\hat{x}$ are scalars.

- (a) Write $\hat{x}$ directly in terms of w_e , w_d , and x .
- (b) Derive $\frac{\partial L}{\partial w_d}$.
- (c) Derive $\frac{\partial L}{\partial w_e}$.
- (d) Use $x = 2$, $w_e = 0.5$, $w_d = 0.4$, and learning rate $\eta = 0.1$. Compute one gradient descent update for w_e and w_d .

Solution 3.

- (a) Since $z = w_e x$ and $\hat{x} = w_d z$,

$$\hat{x} = w_d w_e x.$$

- (b) Using the chain rule,

$$\frac{\partial L}{\partial w_d} = \frac{\partial L}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial w_d} = (\hat{x} - x)z.$$

- (c) Again using the chain rule,

$$\frac{\partial L}{\partial w_e} = \frac{\partial L}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial z} \frac{\partial z}{\partial w_e} = (\hat{x} - x)w_d x.$$

- (d) First compute the forward pass:

$$z = w_e x = 0.5 \cdot 2 = 1, \quad \hat{x} = w_d z = 0.4 \cdot 1 = 0.4.$$

The reconstruction error is

$$\hat{x} - x = 0.4 - 2 = -1.6.$$

The gradients are

$$\begin{aligned} \frac{\partial L}{\partial w_d} &= (-1.6)(1) = -1.6, \\ \frac{\partial L}{\partial w_e} &= (-1.6)(0.4)(2) = -1.28. \end{aligned}$$

The updates are

$$\begin{aligned} w_d &\leftarrow 0.4 - 0.1(-1.6) = 0.56, \\ w_e &\leftarrow 0.5 - 0.1(-1.28) = 0.628. \end{aligned}$$

Both weights increase because the reconstruction 0.4 is too small compared with the target 2.

Exercise 4.

A de-noising auto-encoder receives a corrupted input but is trained to reconstruct the clean input. Suppose the clean input is

$$x = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix},$$

and the corrupted input is

$$\tilde{x} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

After feeding $\tilde{x}$ through the auto-encoder, the model outputs

$$\hat{x} = \begin{bmatrix} 0.9 \\ 0.7 \\ 0.1 \\ 0.2 \end{bmatrix}.$$

Use mean squared error.

- (a) In a de-noising auto-encoder, should the loss compare $\hat{x}$ with x or with $\tilde{x}$?
- (b) Compute the de-noising reconstruction loss.
- (c) Compute the loss that would be obtained if we incorrectly compared $\hat{x}$ with $\tilde{x}$.
- (d) Explain why adding noise during training can produce more robust features.

Solution 4.

(a) The loss should compare $\hat{x}$ with the clean input x , not with the corrupted input $\tilde{x}$. The goal is to recover the original signal from a noisy version.

(b) The de-noising loss is

$$\begin{aligned} \text{MSE}(\hat{x}, x) &= \frac{1}{4} [(0.9 - 1)^2 + (0.7 - 1)^2 + (0.1 - 0)^2 + (0.2 - 0)^2] \\ &= \frac{1}{4} (0.01 + 0.09 + 0.01 + 0.04) \\ &= \frac{0.15}{4} = 0.0375. \end{aligned}$$

(c) If we incorrectly compared to the corrupted input,

$$\begin{aligned} \text{MSE}(\hat{x}, \tilde{x}) &= \frac{1}{4} [(0.9 - 1)^2 + (0.7 - 0)^2 + (0.1 - 0)^2 + (0.2 - 0)^2] \\ &= \frac{1}{4} (0.01 + 0.49 + 0.01 + 0.04) \\ &= \frac{0.55}{4} = 0.1375. \end{aligned}$$

(d) If the model only learns to copy inputs, it may learn brittle features. If the model sees corrupted versions and must reconstruct clean versions, it has to learn stable patterns in the data. For images this may mean edges, shapes, and textures. For tabular data this may mean typical correlations between features. The noise forces the model to use context instead of memorizing one coordinate at a time.

Exercise 5.

An auto-encoder is trained only on normal data. At test time, the reconstruction error is used as an anomaly score:

$$s(x) = \text{MSE}(x, \hat{x}) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2.$$

A point is classified as an anomaly if $s(x) > T$, where $T = 0.05$.

Consider the two test points below:

$$x^{(1)} = \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad \hat{x}^{(1)} = \begin{bmatrix} 0.95 \\ 0.90 \\ 0.05 \\ 0.10 \end{bmatrix},$$

$$x^{(2)} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad \hat{x}^{(2)} = \begin{bmatrix} 0.40 \\ 0.20 \\ 0.70 \\ 0.30 \end{bmatrix}.$$

- (a) Compute the anomaly score for each test point.
- (b) Decide whether each test point is normal or anomalous. Briefly explain the reasoning.

Solution 5.

(a) For the first test point,

$$\begin{aligned} s(x^{(1)}) &= \frac{1}{4} [(1 - 0.95)^2 + (1 - 0.90)^2 + (0 - 0.05)^2 + (0 - 0.10)^2] \\ &= \frac{1}{4} (0.0025 + 0.01 + 0.0025 + 0.01) \\ &= \frac{0.025}{4} = 0.00625. \end{aligned}$$

For the second test point,

$$\begin{aligned} s(x^{(2)}) &= \frac{1}{4} [(1 - 0.40)^2 + (0 - 0.20)^2 + (1 - 0.70)^2 + (0 - 0.30)^2] \\ &= \frac{1}{4} (0.36 + 0.04 + 0.09 + 0.09) \\ &= \frac{0.58}{4} = 0.145. \end{aligned}$$

(b) Since $s(x^{(1)}) = 0.00625 < T = 0.05$, the first point is classified as normal. Since $s(x^{(2)}) = 0.145 > T = 0.05$, the second point is classified as anomalous. The reasoning is that a normal point should be reconstructed well by an auto-encoder trained on normal data, while an unusual point should have a larger reconstruction error.